2026 IEEE 6th International Conference on Smart Information Systems and Technologies (SIST)
13-15 May, 2026, Astana, Kazakhstan

Customizable methodology for structured analysis of software within isolated computational environments to secure critical IT infrastructures

1st Noyan Tendikov
School of Cybersecurity
Astana IT University
Astana, Kazakhstan
242710@astanait.edu.kz
ORCID: 0009-0009-7251-8830

2nd Rostyslav Lisnevskyi
School of Cybersecurity
Astana IT University
Astana, Kazakhstan
rostyslav.lisnevskyi@astanait.edu.kz
ORCID: 0000-0002-9006-6366

3rd Leila Rzayeva
School of Cybersecurity
Astana IT University
Astana, Kazakhstan
l.rzayeva@astanait.edu.kz
ORCID: 0000-0002-3382-4685

***Abstract*—Critical IT infrastructures are vulnerable to indirect attacks, in which malicious actors affect them through their software dependencies and internal components. This research aims to explore this issue by focusing on software composition and its unification, extending the current approaches through a conceptual framework for analyzing software in specialized isolated environments. Practical results in a simulated environment demonstrate the dynamics of compromised package propagation among various actors. Our findings suggest that adopting restrictive quality gates and isolated analysis environments is a necessary step to shift toward proactive prevention, effectively controlling the spread of malicious software and catching complex threats before they reach a production state.**

***Keywords*—software supply chain; infrastructure-as-code; critical IT infrastructure; software composition analysis; software dependencies; software bill of materials; sandboxing; malware and binary analysis; digital twins; intelligent agents**

I. Introduction

Modern cyber conflicts have escalated into a condition of constant cyberwar, driven by countless threats. Unlike traditional military conflicts, this warfare is taking place within a digital space and often goes unnoticed by the general public due to their lack of awareness and competence [1]. The current policy of major states is dictated by the principle of "peace through strength", prioritizing the rule of power over ethical considerations and international law. It leads to an intensified arms race between multiple actors, such as government agencies, armies, corporations, and private groups. Under these circumstances of ongoing confrontations between defenders and attackers, ensuring security is only a continuous process rather than an absolute guarantee [2]. So, the industry of information technology (IT) should be engaged in systematic risk assessment and apply reactive patches as problems emerge, because the number of potential threats and vulnerabilities is inherently unknown and cannot be fully identified or eliminated upfront. Moreover, the increasing rate of automation and digitalization exposes associated risks for critical infrastructure, where their compromise could create dangerous situations and collateral damage for citizens [3, pp. 1–2]. For example, CISA (the Cybersecurity and Infrastructure Security Agency of the USA) adopted a classification that identifies 16 critical infrastructure sectors, including the context of IT infrastructures [4, pp. 6–7]. This sector is composed of corresponding services and assets (e.g., maintenance personnel, electricity, networks, equipment, hardware, software), providing an operational basis or computational environment to utilize for information systems (IS) [5].

Fundamentally, attacks targeting critical IT infrastructure are driven by diverse motives and reasons, overlapping with other industries [6]. Historical events have already shown that conflicts of national interests might trigger a security breach on highly protected facilities (e.g., nuclear power plants) [7]. For instance, Riggs et al. examined statistics on global cyberattacks against critical infrastructures since 2006 and developed a prediction model that shows exponential growth of incidents with the military sector being the most frequently targeted [8]. Another factor that may escalate the situation is the growing trend around generative artificial intelligence (AI), which increases the demand on data centers, stimulating competitors and other third parties to compromise or abuse that computational power for their own benefit [9]. Additionally, the increasing interconnectivity of IS, especially through Internet of Things (IoT) devices, causes fragility by exposing multiple points of failure and allows malicious actors or even device manufacturers to gain remote access to them [10], [11].

Therefore, this study attempts to construct security measures for the IT infrastructure, emphasizing the process of collecting and accounting of its internal resources or components for reliability and early insider threat prevention. Firstly, the literature review covers the historical development of IT infrastructure, complexity management, and issues with software composition, addressing current needs and challenges in the field. Secondly, the methodology specifies the hypotheses and provides a framework for conducting customizable analyses within sandboxed environments. Thirdly, the results part examines features of the proposed solution. Lastly, the conclusion summarizes findings and potential drafts for future work.

II. Literature Review

A. *Historical development of IT infrastructure*

The computing ecosystem and its service models have evolved gradually to on-premises and cloud paradigms, becoming increasingly commercialized to meet customer demands and scaling issues, as broadly represented in Figure 1.

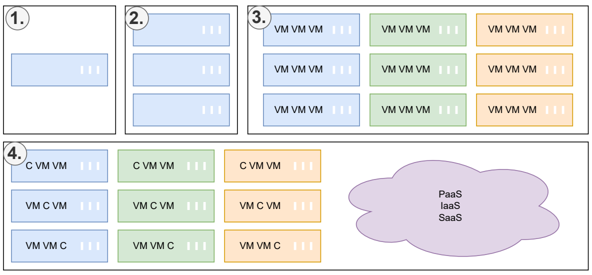

Figure 1. Developmental milestones of computing environments (VM - virtual machine; C - container; PaaS, IaaS, SaaS - external services): (1) Mainframe. (2) Cluster of servers. (3) Geographically distributed servers with virtualization (visually represented by color difference). (4) Hybrid approach with cloud services.

Initially, there were only mainframes that required dedicated teams for maintenance and performed at a highly mechanical level, so their bulkiness and slow speed enforced considerations for the adaptation of distributed computations [12, Sec. 3.1]. Fortunately, improvements in IT manufacturing significantly contributed to the emergence of dedicated servers that could be networked together in a modular way to provide scalability. Following this trend, virtualization moved beyond bare-metal deployments by allowing isolation layers to securely run multiple virtual machines (VMs) on shared hardware to significantly enhance the efficiency of resource utilization [13]. Eventually, increasing demand for international connectivity and economic globalization naturally formed large-scale data centers with a network of geographically distributed servers in order to optimize content delivery and processing [12, pp. 9–11].

Nowadays, organizations can offload many responsibilities and parts of their IT infrastructure to cloud and service providers which abstract physical machines and offer instant on-demand resource provisioning and management, thereby reducing operational burdens via "as-a-service" models with specialized offerings [14]. Software engineers often take that for granted, underestimating the interdependence of software and hardware sides for facilitating the execution of general-purpose tasks within multiple layers of abstraction. Such a service-based approach for outsourcing still requires integrating and coordinating operations over delegated parts (e.g., security policies, privacy compliance, and external monitoring). Moreover, certain strategic systems (i.e., governmental and military) and their data are prohibited from being hosted in the cloud due to secrecy laws and regional restrictions [15], [16].

B. *Complexity management with codification and modeling*

As the sector of IT infrastructure has expanded, its complexity and associated challenges have emerged accordingly. Nonetheless, this complexity has become the primary factor for the process automation against manual, repetitive, and impractical operations. Therefore, automation and orchestration are no longer optional - they are critical for leveraging fully scalable and reliable systems from a strategic perspective. One of the natural strategies for complexity management is code. The process of codification captures procedures, routines, or algorithms of actions in a textual format [17, pp. 1–2]. For example, Chuprikov et al. propose specifying security policies in IT systems in the form of domain specific language (DSL) that can be run within an execution engine [18]. Codification also established the paradigm of Infrastructure-as-Code (IaC) which is focused on specifying the state and content of IT infrastructures to enable reproducibility (e.g., provisioning, replication, copying, scaling) and modification (with emphasis on transparency and maintainability), as well as management and maintenance by self-documenting code or files that incorporate defined metadata, configurations, and procedures [19]. It minimizes human errors and establishes immutable infrastructure with the phoenix model, allowing any server instance to be recreated, replaced, updated, or destroyed from code without losing critical configurations. This approach eliminates the existence of snowflake servers, which are deviated systems with configuration drift and unpredictability due to their manual modification and absence of documentation [20, Ch. 2].

IaC operations are managed by system administrators, DevOps teams (Development and Operations), SREs (Site Reliability Engineers), or IT professionals. They utilize IaC toolkits based on a spectrum of paradigms:

- Imperative - describes the process of achieving the target state via scripting languages and native shell scripts.
- Declarative - describes the final target state using definition files, configurations, and manifests.

Usually, declarative ones (e.g., Terraform, Ansible, Nix) are implemented internally via general-purpose languages, but using configurations as application programming interfaces (APIs) [20, Ch. 4]. For example, containerization tools like Docker use Dockerfiles to define application dependencies and versions, while the runtime builds a container with artifacts according to specified requirements, as shown in Figure 2.

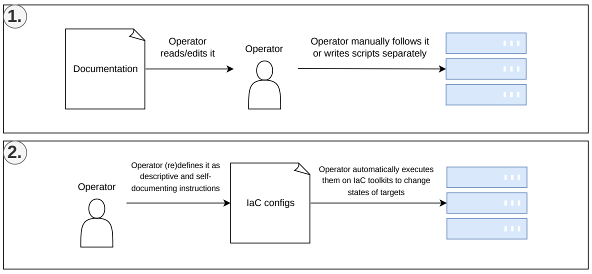

Figure 2. Processes of applying changes to servers: (1) Manually or with simple scripts by following separate documentation. (2) IaC pipeline as a self-documenting system.

Any IaC file should be proceeded through the continuous integration and continuous delivery (CI/CD) pipeline, which maintains safe deployment by runtime verification tests within simulated conditions for dynamic analysis and automated scans with validation checks of the codebase for static analysis [21, pp. 9–10]. According to Saavedra et al., incorporating intermediate representation (IR) and source-to-source translation into IaC static analysis facilitates the development of language-agnostic methods that can be ported to any IaC format [22]. So, IT specialists emphasize that IaC codebases should be maintained in accordance with secure coding standards on a regular basis (e.g., ISO/IEC series, NIST frameworks, etc.) [23].

Sandobalin et al. examined the advisability of using IaC modeling tools by training control groups on Argon and Ansible [24]. The research showed that Argon, as a visual toolkit, is easier to learn and can abstract complexity as a thin layer above existing IaC configuration formats. Thus, allowing users to simply draw their IT architecture, while the underlying system automatically generates a specific configuration for deployment. It implies that complexity management and computational thinking also extend to graphical and other representations of information. Diagrams and visualizations provide two-dimensional or multidimensional representations, enabling more structured modes of reasoning, whereas text is inherently a linear representation of information. Certainly, these forms should not be regarded as mutually exclusive, rather they synergize with each other and are interchangeable at different levels of abstraction.

In this regard, Model Driven Engineering (MDE) methodology proposes treating documentation, specifications, and model definitions as primary and reference artifacts describing behavior and components of a system, from which code generation toolkits can facilitate transformation between them (e.g., into source code templates in any programming language or pseudocode) in order to achieve synchronization [25]. Firstly, it bridges the gap between developers and business-oriented specialists by demonstrating certain domain-specific problems and rules, so they can focus more on business logic. For instance, Alfonso et al. developed the BESSER platform based on MDE, which provides a canvas editor and multiple design notations like UML (Unified Modeling Language) for generating backend applications from diagrams [26]. Such toolkits usually support the development of reliable prototypes and concepts, but they are not intended for high-load or performance-critical systems. Secondly, MDE allows structures to dynamically adapt to emerging requirements instead of manually rewriting boilerplate code and thinking about platform-specific implementations, so the development process is optimized. For example, it is applicable for automatic propagation of components changes and migration between layers instead of repetitive manual transformations, conversions, and modifications [27]. Additionally, AI in the form of language models enables rapid code generation from specifications as detailed prompts and instructions [28], [29]. Although generative language models can cause challenges due to hallucinations and their stochastic nature, it might be partially mitigated through enforcing strict rules, validation mechanisms, and multiple series of output refinements [30].

C. Resource management and accounting

OWASP (Open Worldwide Application Security Project) identified 10 major risks for IT infrastructure of which 5 are directly related to IT resources accounting [31]: "Outdated Software", "Insufficient Threat Detection", "Insecure Resource and User Management", "Insecure Access to Resources and Management Components", "Insufficient Asset Management and Documentation". This fact clearly emphasizes that proper management of all system components and organizational knowledge is mandatory. In this context, there is already a structured approach for collecting and accounting software components, such as software composition analysis (SCA), which defines common specifications and governance principles [32]. It facilitated the establishment of a widely adopted format for software bill of materials (SBOM) with its multiple variations that allows to reduce a total vulnerability response time [33], [34]. Other progressive solutions can assist in the same task by collecting data from a running production instance to subsequently generate recreatable configurations and identical images/snapshots [35, pp. 5–7]. Usually, such procedures are combined with version control systems (VCS) that store and track versions of all project files, allowing teams to have: process of collaborative development, a single source of truth, and well-defined compliance by localizing areas of responsibility [36]. Particularly, connections and links between system components form "dependencies" which have direct and indirect involvement on certain functionalities and qualities of the base system. As demonstrated in Figure 3, OS (operating system) can be an illustration of a complex system that requires the correct operation of its components providing emergent qualities.

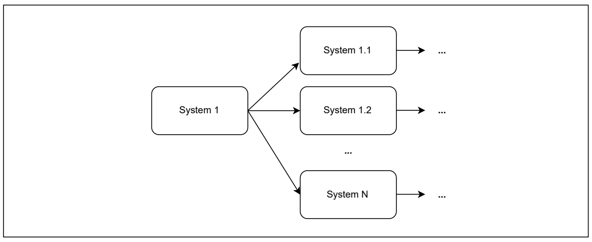

Figure 3. An abstract complex system consists of multiple underlying operational components and subsystems.

According to statistics of the European Cyber Security Organisation (ECSO), solution-specific logic often represents only about 10% or less of an application's total code, while the remainder is largely made up of third-party dependencies such as software (module, package, library), service (requests to external systems and network API), infrastructure (like drivers and execution platform) [21, p. 8]. Given a script in the Python programming language, the internal dependency in this case is the self-contained code, while the external dependency is everything that was created by third-party such as the implementation of standard library, interpreter/compiler, specification, OS, or platform-specific requirements, and so on, as shown on Figure 4.

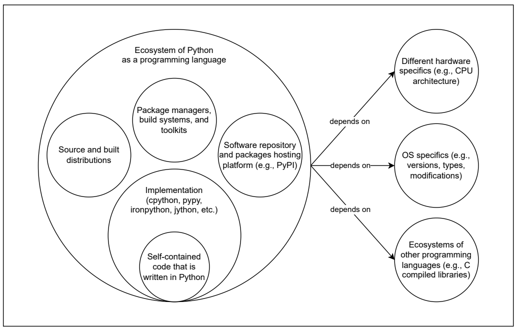

Figure 4. Ecosystem of the Python programming language from the perspective of dependencies. Adapted from [21, p. 8].

From a broader perspective, the modern economy is service-oriented and highly interconnected in the global supply chain, where everyone relies on each other, as represented in Figure 5. Hence, engineers can mitigate issues of resource constraints (e.g., schedule, personnel, risks, requirements, finance, and qualifications) by integrating external dependencies and reusing code as outsourcing for common problems during the implementation and maintenance of IT products.

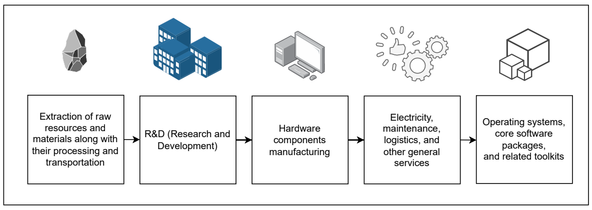

Figure 5. Simplified example of the global supply chain in the IT context.

D. *Compromise and trust in security*

On the other hand, any risks and failures in a single link can cause a cascading effect across the entire chain, which is known as a supply chain attack. This attack vector exposes the ability to implicitly or explicitly influence on any IT infrastructure by leveraging direct and indirect (i.e., transitive) dependencies, invoking instability and fragility [37]. It can be caused by any potential way of external influence, including unintentional flaws and defects, intentional threats like malicious software (malware), or destabilizing factors like breaking API changes and internal modifications [38]. Inadequately governed dependency practices within a team can increase the system's exposure to software supply chain attacks. Accordingly, it is necessary to be familiar with various approaches to dependency management, such as:

- Manual ("vendoring") - by copying an external dependency directly into the codebase in order to maintain and modify it independently.
- Automatic - dependency/package managers as separate software system that is responsible for retrieving, handling, updating, and executing additional lifecycle operations on project dependencies.

These techniques might be combined together with build specification files (e.g., CMake, Make, Setuptools). Although some practitioners argue that the manual approach provides a more detailed understanding of dependency contents (source and distributions) and greater control (updating only explicitly), both of them are equally susceptible to supply chain attacks due to propagation of transitive dependencies. Consequently, dependencies that are not explicitly pinned should not be relied upon, since they may disappear or be replaced within the direct dependencies from which they originate.

Adversaries actively exploit such kind of characteristics and behaviors. Someone can abuse identity confusion by impersonating existing packages (via name similarity, description, or other metadata), claiming unregistered package names, and performing additional actions intended to mislead tools or users [39]. Therefore, it is essential to understand the underlying mechanisms, such as dependency resolution and their metadata parsing. From a theoretical standpoint, dependency resolution is a variation on the topic of the constraint satisfaction problem where the objective is to satisfy a predefined set of conditions and rules. It is proven to be an NP-hard problem for which no global optimal solution exists. Figure 6 indicates the process of dependency management in the Python ecosystem via the PIP package manager and the PyPI repository that uses backtracking as a dependency resolution algorithm. On the other hand, NodeJS, with its NPM package manager, handles the same process via workarounds by downloading multiple duplicate dependencies with different versions and requirements, which affects the large size of the node_modules directory.

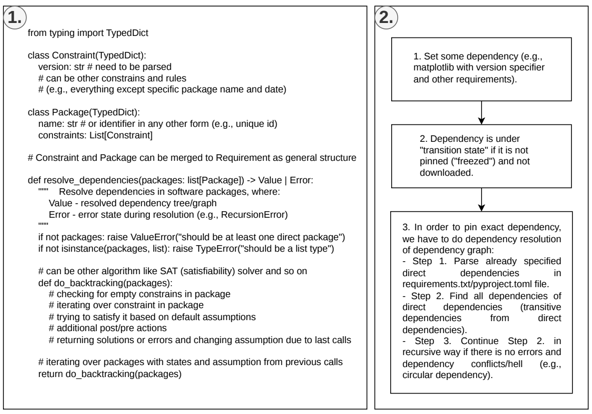

Figure 6. Process of dependency management: (1) Dependency resolution in Python-like pseudocode. (2) Dependency parsing and pinning.

Constructing the dependency tree or graph becomes increasingly complex with each additional dependency and constraint (version requirements or other ecosystem-specific rules), as illustrated in Figure 7. Firstly, version constraints restrict the allowable range of package versions from the initial published version to the latest one according to the versioning format. Such versions basically represent a variation of the same entity. Secondly, dependency groups delimit or expand the number of required packages, because some components can be mandatory while others remain optional. Thirdly, Figure 7 represents only a basic example with a limited number of dependencies. In real projects, the number of dependencies and constraints is significantly larger, leading to dependency conflicts ("dependency hell") characterized by cross-references, circular or recursive relations, loops, duplicates, and similar pathological structures. These situations might be resolved, for example, by flattening the dependency tree into a more linear structure.

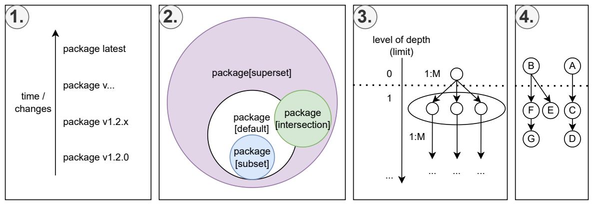

Figure 7. Elements of dependency theory: (1) Version constraints. (2) Dependency groups (concept from set theory). (3) and (4) Nested dependency graph (concept from graph theory) with direct and transitive ones (e.g., "1:M" as "one to many" relationships and other relations between them).

Many software repositories prioritize functionality over security, often justifying this by claiming that they do not wish to restrict access for a community that is "open for everyone", mistakenly assuming that security represents an unnecessary overhead [40]. As a result, they expose themselves and others to the risk that anyone may maliciously influence ecosystems. So, users are left to manage the external chaos on their own. Despite rising SCA adoption by developers, multiple researchers indicate that supply chain vulnerabilities and incidents continue to increase, and the remediation process often remains slow [41], [42]. Moreover, Alfadel et al. found that, in the Python ecosystem, it takes a median of up to 4 months to fix all occurrences of a security vulnerability in the PyPI repository [43]. Existing market solutions cannot be regarded as definitive protections against supply chain attacks, as experts identified several additional drawbacks and limitations of current SCA tools [44], [45], [46]:

- focusing more on static analysis rather than on dynamic analysis during runtime execution;
- comprehensive deep scanning of transitive dependencies is often missing;
- insufficient integration with AI and data analytics-based systems.

Furthermore, obtaining reliable data about software remains challenging due to its various distribution formats and categories:

- white box - fully known content (e.g., open source);
- gray box - partially known content (e.g., leaked information, rumors, reverse engineering, insider knowledge, alternative products, or forks);
- black box - unknown content (e.g., functionality behind API like SaaS, proprietary and closed-source software).

General SCA tools typically focus only on the white box approach by gathering already indexed vulnerabilities or components, thereby leaving critical gaps in threat visibility and transparency. Evidently, software analysis and detection/identification for other categories (both malware and legitimate) are inherently more complex and complicated due to their specifics, because anti-analysis techniques are used to protect such software and intellectual property, making component-level inspection difficult or reasonably impossible [47]. Although services like VirusTotal or similar antivirus platforms scan uploaded untrusted artifacts against numerous antivirus engines and databases to protect the corporate or personal sector, they cannot cover all possible forms of malware. For example, on platforms such as Google Play or similar repositories, there are many instances of hidden malware or fully undetectable malware (FUD) that have successfully bypassed the verification stage. Unfortunately, there is no universal solution or method against that, since software protection is constantly evolving. Therefore, it is mandatory to conduct additional reconnaissance and data gathering of existing samples and resources via well-known or standard deconstruction methods. In the realm of malware analysis, objects that are stored or set for analysis are often termed as "IoCs" (Indicators of Compromise), "artifacts", or simply "samples" [48]. For example, these include entities such as IP addresses, URLs (links), DNS (domain names), virus signatures, hashes, files, and more. It is essential to clarify that the previously mentioned malware analysis is not limited solely to analyzing malware, and the term "malware" itself is quite vague/broad. Essentially, malware is any software which is considered potentially harmful or could lead to a security breach, depending on the specifics of our threat modeling with a predefined set of malicious characteristics and behavior. Malware analysis itself is typically categorized into four main types, which can be combined in various ways:

- Static analysis focuses on examining the malware without executing it, which involves inspecting code, file structure, metadata, strings, and other non-executable aspects.
- Dynamic analysis, on the other hand, observes the malware in action by executing it in a controlled sandbox environment. This type of analysis monitors real-time / runtime behavior, such as network activity, file modifications, and registry changes, and is sometimes called "behavioral analysis" [49].
- Manual analysis involves hands-on inspection of code or source by an analyst, allowing for in-depth, targeted evaluation.
- Automated analysis, on the other hand, leverages tools to systematically assess and classify malware or provide preliminary insights.

The fragmented software ecosystem worsens this situation, making accountability for the many packages and their adherence to standards unclear [50]. For instance, Mirakhorli et al. reviewed 84 available SBOM management toolkits across multiple programming language ecosystems, which demonstrates the lack of unification and significant fragmentation in current solutions [51]. Even a software package that is trusted by the community cannot offer proper guarantees, because mainstream software licenses clearly mention that their products are distributed "as is" condition [52]. Experts propose different solutions for the current status quo, ranging from weak to radical measures. Firstly, there should be an introduction of increased mandatory and strict basic requirements (baselines) at the platform level with compliance from end user side. This could reduce the number of attacks and malicious code due to high standards and improved quality. It also reflects the principle of "secure by default", where secure policies and good default configurations should become common for many, and not just the exception reserved for professionals:

- Mandatory oversight and analysis before package publication and subsequently with every update/change. For example, the establishment of a moderation system where the community will naturally report and react to the appearance of malicious packages. Such movements as bug bounty programs can provide payment to the community for finding bugs to establish natural vulnerability research and embrace ethical hacking over illegal activities [53], [54]. Also, there is a productive case within Astana IT University where they regularly implement CTF (capture the flag) and cyberpolygons to train security engineers, thereby validating skills and preparing professional specialists for organizations [55].
- Policies for temporary blocking or issuing "call down" periods between user actions during anomalous activity (e.g., IP address change, password or specific token renewal, recent publications, and so on).
- Movement towards slowing down of overproduction and starting to "recycle waste" by rethinking legacy systems or abandoned ideas (re-evaluating them, finding patterns, and recognizing repetition) [56]. Perhaps, developers should also strive to regularly improve the standard library and make it better with "batteries included" in the official distribution. Otherwise, its position will be taken by many competing dependencies covering the exact same specific topic, thereby generating a high degree of entropy.
- Designing systems in a way that they are "secure by design" in their foundation means that security and safety are built in initially, rather than being an additional element. This approach dictates that engineers strive to place security and features at least on the same level of value during the development process [57]. It eliminates specific categories of vulnerabilities, restricts/limits

malware, and forms a multilayered security / defense-in-depth approach [58].

- Next-generation and future-proof systems that are comprehensive to the extent that they can self-analyze and self-improve, making them more reliable and less fragile. For instance, Berhe et al. showed that companies are compelled to have a mitigation plan and a regular budget for automated software dependency recovery or manual actions such as replacing it with alternatives, upgrading it, or reverting to a previous release [59]. Ideally, these issues could be mitigated by well-decomposed systems that might still continue to function in such critical cases or have internal recovery mechanisms ("self-healing") for the failure of individual components only up to a certain limit [60].

Secondly, the software market should pay more attention and show greater concern for formal regulatory mechanisms to increase trust and responsibility in the community, such as certification, auditing, and transparent processes, along with initiatives aimed at improving standards. Sammak et al. attended an interview among a small subset of experienced developers, which revealed that the majority of respondents are unfamiliar with details of software supply chain management or have a superficial understanding due to the absence of proper training on this topic for a broader audience [61]. Thirdly, popular or operation-critical software products have a strong influence on the whole landscape, so compromising them can undermine not just dependent infrastructure but also the global contracts of security, such as the cryptography stack [62]. For example, self-replicating malware or a virus might compromise a significant portion of the IT ecosystem by ensuring that any code compiled on an infected hardware platform, compiler, or OS becomes compromised as well [63]. This is due to the fact that definition files are merely instructions without guarantees that our intentions will be correctly expressed and transformed with preserved semantics in the produced output (e.g., machine code, process, and other representations) by an intermediary system (compiler, interpreter, execution environment), as shown in Figure 8.

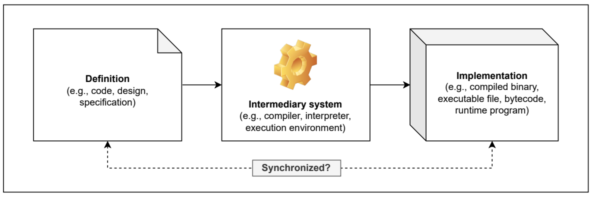

Figure 8. An intermediary system between design and implementation.

Furthermore, conducting a compromise assessment or software evaluation for the produced product is complicated, because infected reverse engineering tools (e.g., debugger, decompiler, disassembler) or even their clean versions running in a compromised environment might hide the infected parts and backdoors as blind spots in the software in order to remain undetected. Due to the high stakes, it is crucial to determine whom to trust and based on what principles to minimize such risks beforehand. Researchers Singer and Bishop propose viewing trust in the form of a relationship model that makes us vulnerable because we rely and depend on a third party without any doubt due to assumptions and distractions, considering them trustworthy [64]. It means that our inherent tolerance to possible flaws/mistakes due to the factor of trust affects on our cybersecurity posture. They also suggest applying threat modeling and risk analysis before and during the design stage in the development lifecycle to minimize the factor of trust. Hence, trust should not be considered as a static and accurate indicator, but rather it is a dynamic probability of being trusted by someone else and vice versa, where any mistake from both parties leads to its loss. For example, one of the parties (recipient or sender) can at any time disrupt data transmission and its correctness by intentionally beginning not to adhere to the given standard protocols and interaction interfaces.

For these reasons, the National Institute of Standards and Technology (NIST) developed the concept of Zero Trust Architecture (ZTA), in which trust requires continuous verification [65]. Babenko et al. clearly demonstrated that empirical data on ZTA implementation correlates with a lower incident rate of insider threats [66]. Organizations can adopt this by following the key principles listed below:

- Continuous evaluation of all actions and actors instead of granting full access within a protected perimeter. This means the environment itself does not affect how we trust someone/something, assuming operating in a hostile environment [65, Ch. 2.2].
- Eliminating implicit trust in favor of explicit trust using a trust algorithm as a method for verifying access or credibility with transactional properties [65, Ch. 3.3].
- Segmentation of the network and systems into atomic cells, which theoretically fragments the attack surface (e.g., a large monolith will expose more data than individual microservices), as visualized on Figure 9 [67].
- Least privileges of access and minimizing the amount of dependencies via strict comprehensive assessments of vendors or suppliers [68]. On top of this, it is recommended to apply DevSecOps practices for the automation of software building, testing, and the overall "shift-left" movement for the early identification of problems in information security [69].
- Security as a dynamic process and adaptive system is achieved through the continuous

collection and analysis of data/statistics (resources, assets, incidents, etc.). In particular, Mushtaq et al. note the importance of compliance with GDPR (General Data Protection Regulation), explainable AI, and anomaly detection to maintain transparency in ZTA [70].

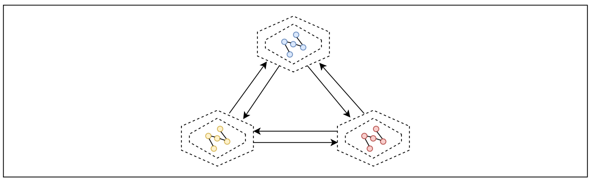

Figure 9. Network segmentation in ZTA, where each segment consists of multiple defense layers and granular microservices.

One example of ZTA implementation is decentralized systems (e.g., blockchain), which contain zero-knowledge proofs and consensus algorithms such as "proof of work" and others that guarantee trust based on a defined trust factor (e.g., computational power, number of devices/actors, amount of specific tokens) [71]. Furthermore, there is a concept of "trustless" in the same sphere that further reinforces ZTA and trust management approaches in distributed systems. Some studies even consider the potential for applying blockchain as a version control system due to its data integrity and non-repudiation characteristics [72].

Overall, the current landscape of security for critical IT infrastructure is broad and continues to evolve alongside the increasing sophistication of cyberattacks. It is now evident that there is a significant need for more robust dynamic analysis of software and its underlying components, as static source code analysis and standard tooling are insufficient. Future large-scale or critical incidents will likely serve as catalysts, compelling the global community to fundamentally rethink and restructure prevention strategies. So, the industry is going to shift toward proactive prevention, the establishment of comprehensive security protocols, and the rigorous verification and analysis of IS.

III. Methodology

A. *Proposal concept and paradigm shift*

Therefore, this study proposes to adopt principles from existing malware and binary analysis tools with digital forensics, which can be integrated into the process of collecting and analyzing both malware and legitimate components. These toolkits can effectively complement modern SCA approaches and enable a paradigm shift, as illustrated in Figure 10. It redirects the purpose of existing solutions toward a different target audience and analytical focus from regular end users to developers. Although this may appear to be a minor shift, it is both critical to have and straightforward to integrate.

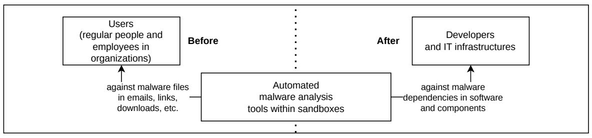

Figure 10. Paradigm shift by applying principles from malware and binary analysis for SCA-related tasks.

These systems are essential tools for a range of specialists, including reverse engineers, malware researchers, RnD (research and development) analysts, and SOC (security operations center) analysts, who use them to gain in-depth insights into malware behavior and potential threats. Companies often deploy these solutions in various configurations, including on-premises and SaaS models, as shown in Figure 11.

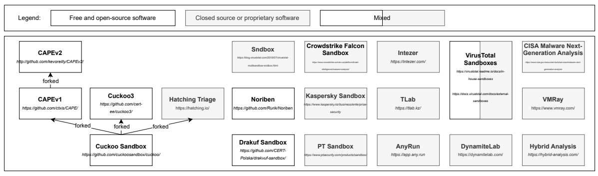

Figure 11. Landscape of well-known solutions for security analysis and sandboxing. On-premises deployment is local within the organization due to data confidentiality and security requirements. SaaS platforms contribute to broader landscape by aggregating and analyzing threat data from multiple worldwide sources.

Sandboxing approach is validated by NIST for ZTA, where it is recommended to manage software examination within isolated execution environments before allowing to add or change of any component into the IT infrastructure [65, Ch. 3.2.4]. However, only passive analysis with sandboxes and honeypots is limited, because security engineers cannot predict some situations due to a small subset of analyzed artifacts and insufficient data. Also, the behavior observed in a sandboxed environment is not always representative of the real system. Therefore, organizations should strive to simulate the original infrastructure and everything in it to be exactly precise to the production instance in order to obtain more correct and realistic results in general, as demonstrated in Figure 12.

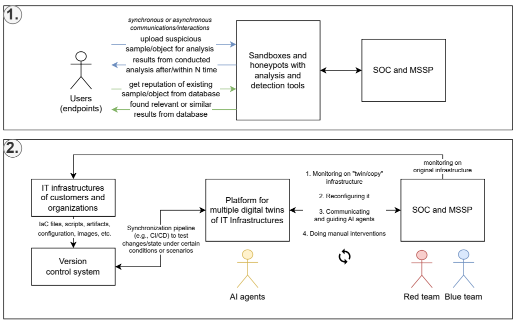

Figure 12. Transition to cycle of continuous security (SOC -

security operations center; MSSP - managed security service provider): (1) Passive analysis with sandboxes and honeypots. (2) Reactive analysis with digital twins of IT infrastructures with AI agents.

Full-scale digital twins allow to create copies of specific workstations up to entire large systems as complex real-time simulations. They might significantly reduce the time and cost of testing and validation, making the process truly continuous and assisting in identifying critical problems before publication into production. In essence, such an analysis laboratory establishes a validation and feedback loop against constantly expanding attack surfaces, where security specialists can simulate multiple attack scenarios and gather all data from the IT infrastructure. It also provides the following features and opportunities:

- Digital hygiene via a mirrored, restricted, and remote environment for isolation and security. This ensures that any incoming untrusted artifacts from external sources, which may pose a hazard, do not create high risks associated with local execution during the parsing, downloading, and executing processes.
- The connection can be secured by an encrypted VPN (virtual private network) tunnel, enabling the safe use of RPC (remote procedure calls), SSH (secure shell) connections, HTTP/HTTPS (hypertext transfer protocol) transfers, and so on. Additionally, the instance is restricted by a firewall to block all unauthorized inbound connections and prevent outbound connections (egress filtering) to mitigate risks such as reverse shells.
- Hardening the environment to ensure a bad actor cannot interfere with or compromise data collection. For example, preventing a scenario where a malicious package contains a script that disrupts the installed monitoring agent or spoofs the collected data.
- Regularly deconstructing and reconstructing IT infrastructure from self-documenting files. This opens opportunities for dynamic and configurable end-to-end testing, what-if analysis, and chaos engineering.
- Immediately destroying an instance and clearing its resources upon completion of a task execution, meaning it is being constantly recreated by a new one as temporary and immutable (similar to short-living or stateless serverless functions). Such a mechanism minimizes the risk of sandbox escape, preventing a malicious actor from pivoting into internal infrastructure through the sandbox environment.
- Portability and compatibility across various targets (device architectures, hardware combinations, OS platforms) with emulation.
- Extensible distributed systems that are made of multiple server instances with virtual machines and containers based on different forms of hosting.

Furthermore, current agentic AI capabilities have reached a significant level through multimodality (a combination of language, vision, audio, etc.), allowing for their orchestration toward specific tasks or the achievement of partial operational autonomy. These intelligent agents could be utilized as lightweight pentesters under customizable scripted scenarios [73]. This approach is effective because even a small specially targeted PoC (proof of concept) script, designed to exploit specific vulnerabilities, can possibly compromise the integrity of highly protected systems. So, it is beyond ordinary brute-forcing or fuzzing due to the flexibility and stochastic behavior of agents. Firstly, the entire infrastructure can be replicated in advance with multiple AI agents (hundreds or more, depending on available resources) deployed to perform large-scale automated testing by actively stress-testing the system and identifying its weak points. In this context, they can conduct user/developer-like interactions or simulate entire communities with unique roles as natural traffic inside a digital twin over an extended period of time, thereby revealing hidden aspects of software during runtime, and finding weak points or unpredictable states of IS within extreme conditions. Secondly, they can be fine-tuned and configured for specific tasks by defining: behavioral patterns, rules, instruction or goals (e.g., according to use cases or user stories), APIs and tools, their simultaneous amount, etc. Obviously, it is a helpful botnet operating for benefit and risk prevention rather than harm to production systems. Thirdly, a similar approach can be extended to other types of simulations by leveraging game engines to regulate actors within the context of real markets, events, distributed systems, locations, and other cases.

B. System modeling and operating mechanisms

There is only a theoretical concept of foundational elements of the solution and their description without strict details on technical aspects and simplification in visualizations in order to be technology-agnostic and portable to other ecosystems. It was decomposed into several parts that align with components and services, ensuring extensibility and customizations. The architectural design is based on the hexagonal architecture because it provides a concept of adapters and ports for replacement or modification of implementations and clear context separation. The driving side represents the system’s entry points and exit points, including regular users and external automated systems that interact with the platform through backend APIs and web interfaces over network connections within a client-server model, as shown in Figure 13.

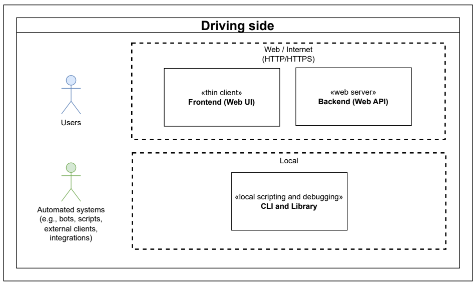

Figure 13. The driving side of the platform.

The internal part is designed as a modular monolith, divided into multiple components or services, as illustrated in Figure 14:

- "Identity provider, user and session management" - maintains all information about users in the platform with their metadata. Generally, "Authentication" and "Authorization" operate alongside each other, forming a comprehensive set of components responsible for user-related security operations: roles or access control based on permissions, login and register handling, access/refresh token management, checking behavior and actions of the user, multi-factor authentication (MFA), and so on.
- "Groups and project system" - holds multiple analyses in which users can collaborate with each other.
- "Moderation system" and "Administration system" - are intended for platform operators, enabling internal system management and response to user requests as needed.
- "Analyses system" - is a critical component that performs all essential operations required for software analysis.

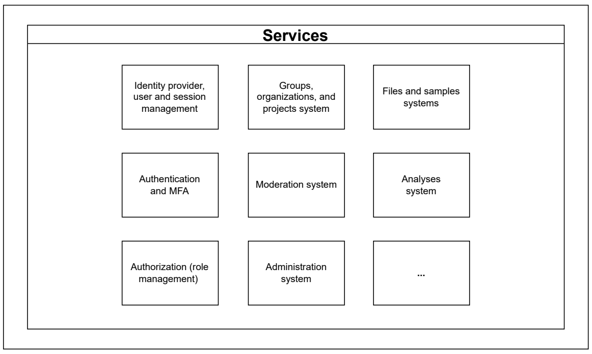

Figure 14. Internal components (i.e., services or subsystems).

Moreover, it has a flexibility aspect that such components can be added, modified, or disabled as needed, which improves security by reducing the attack surface, minimizing system states, decreasing computational complexity, and the factor of combinatorial explosion. The analysis engine in the analysis system also has a plugin and a modding system for fine-grained configuration and operation modes. Firstly, components within the analysis engine are structured according to the actor model, where each component is an object instance, OS process, or container running on the same system or in a distributed environment for scalability. Secondly, some of them are developed for specific use cases related to software packages, container images, computer networks, and other parts of the IT infrastructure. Thirdly, components can be implemented differently but must intercommunicate via JSON-RPC 2.0, and interfaces describe their actions and roles (not limited to the presented set below) operating around the abstraction of "task":

- "Orchestrator" - is responsible for task queuing, scheduling, and overall management operations. It serves as an intermediate layer that receives tasks directly from the analysis system, each with specific requirements that must be resolved before delegation to other components. For example, there are possible types of task requirements such as package (single dependency or a group within one or multiple projects in VCS and archive/compressed format), microservices (container image or layers), and other supported resources. At the same time, “Orchestrator” manages and coordinates the overall execution process, acting as the central communication and control hub, because components cannot begin execution without confirmation or explicit request from Orchestrator. Other components in the analysis engine are a form of "task executors" that perform a specific task. The connection between “Orchestrator” and other components follows a one-to-many relationship, where Orchestrator determines which to use among several available ones or to spawn a specific one.
- “Instancer” - provisions and manages via a common API the required environments (e.g., containers, VMs, specific OS or images, etc.).
- “Collector” - continuously or periodically collects data from created instance (e.g., compares and monitors changes in logs or files, results of build procedure, checking running programs and processes, making image screenshots or recording videos, memory or disk snapshots, constructing a tree of dependencies and processes to manifest file, etc.). Optionally, it is possible to specify a limited scope of data or sources to collect or ignore through regex-based filtering.
- “Interactor” - performs a predefined set of actions or scripted scenarios in an isolated environment to simulate user behavior and trigger malware responses (e.g., interaction with the file system,

launching or installing programs, managing input/output devices, etc.).

- “Parser” - transforms the required data into a unified IR or converts it into a format suitable for “Analyzer”.
- “Analyzer and Reporter” - produce specific verdicts or predictions based on the received data in a structured report (e.g., determining critical components within a dependency tree, performing other specialized analyses using both stochastic and deterministic algorithms).

Figure 15 illustrates the driven side with infrastructural parts and external integrations. Especially, principles of connectivity to artificial instances are represented with agent-based or agentless strategies (similar to "push" and "pull" strategies) in Figure 16.

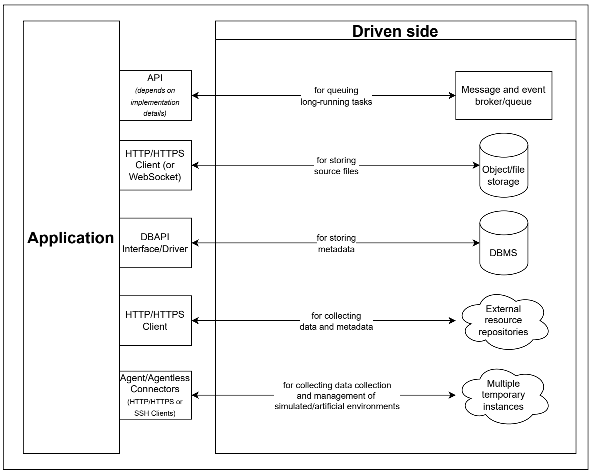

Figure 15. The driven side of the platform.

Both approaches have distinct advantages and disadvantages regarding data transmission, depending on the entity responsible for triggering or handling the transfer. Agent-based methods necessitate the installation of supplemental software on every endpoint, whereas agentless methods allow connection via a single collector (or centralized point). Ideally, organizations should be offered both approaches to allow them to determine which is more suitable for their operational and maintenance needs.

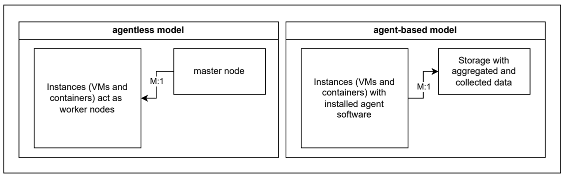

Figure 16. Agent and agent-based strategies where M:1 (“many-to-one”) means a relationship between multiple instances and a single control node.

C. Practical application of the framework within a simulated setup to enhance security capabilities

From a practical standpoint, the proposed approaches can already be integrated into existing systems. For this purpose, we prepared a case study based on the PyPI software repository, which hosts multiple Python packages and operates under a FOSS (Free and Open Source Software) model. Under this model, policies allow any user to use the package manager or other mechanisms to upload, download, and publish packages in the form of source or build distributions. Our methodology is intended to enhance SCA and to establish the principle of a continuous security lifecycle, in which containment is applied prior to publication or modification, rather than immediately accepting changes into the public repository, as illustrated in Figure 17. Figure 18 shows the internal process of the analysis platform in a data-flow diagram. Other details of the analysis system and its engine are shown in Figure 19.

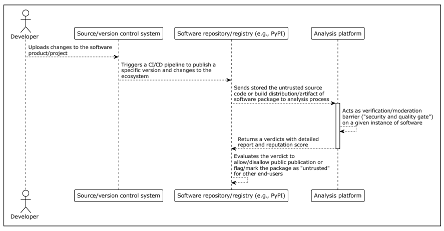

Figure 17. Secure software pipeline by integrating the proposed analysis platform (Part 1 of PyPI example).

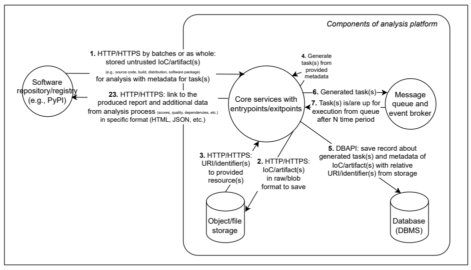

Figure 18. Internal processes of the analysis platform in data-flow diagram (Part 2 of PyPI example).

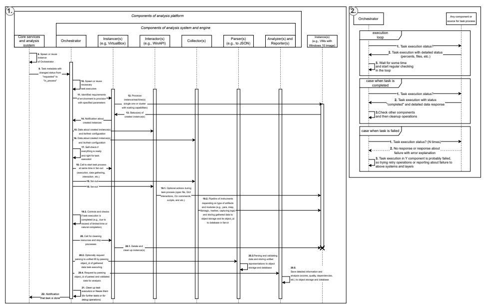

Figure 19. Internal processes of the analysis system and its engine in a sequence diagram (Part 3 of PyPI example).

Therefore, we developed a closed-system computer simulation designed to analyze various dynamics, trends, and potential practical outcomes within a software supply chain. Our primary objective is to conduct a comparative analysis between a reactive approach and a preventive security setup based on a variety of systemic factors. Firstly, we utilized the SimPy library in Python for developing discrete event simulations with defined specific actors and programmed their individual behaviors to accurately model real-world interactions within the repository at a regular rate [74]. Secondly, each simulation step represents one day, with actors performing one action per step. Thirdly, each actor's progression is defined by a specific set of actions, each with a customizable probability of occurrence:

- Developer (i.e., regular user) can install packages with specific version specifiers, manage existing installations, and report suspicious packages to the repository. Additionally, users can perform self-checks to detect potential compromises caused by malicious packages. This detection occurs after a 1-7 day delay, after which the developer has a 50% chance to successfully remediate the compromise and restore their status.
- Package publisher can create new packages and release updated versions within the repository. However, these actions are governed by specific repository constraints, including limits on the number of packages per publisher, total upload volumes, and restrictions on the maximum number of versions allowed.
- Threat actor can release new versions of packages by impersonating and taking control over legitimate publishers, simulating a hijacked account. This allows them to compromise dependencies or launch supply chain attacks designed to infect regular users.

IV. Results and Discussions

A. Results of simulation analysis to compare reactive and preventive approaches

The practical results of the simulation iteration are presented below, alongside specific parameters detailed in Table 1.

TABLE I. Parameters of the Simulation Iteration

Fields	Values
random_seed	42
num_developers	300
num_threat_actors	5
num_publishers	30
max_packages	500 (unique by names)
max_packages_per_publisher	20
max_package_versions	9 (variations of package)
max_steps	365
developer_actions	check_state: 45%, install_package: 20%, change_package: 30%, remove_package: 5%
publish_actions	publish_package: 25%, update_package: 75%
developer_decompromise_success	50%
package_compromisation_reveal_time	range(7, 14)
threat_actor_attack_rate	14

Figures 20 and 21 illustrate the propagation of compromised packages compared to clear packages. Firstly, the simulation shows an unnaturally rapid creation of clear packages. This is due to the lack of rate-limiting for publishers; currently, they perform actions at every step, whereas in reality, these actions occur much less frequently. Secondly, the results for both package types eventually hit a plateau. This happens because the model enforces strict limits on the number of versions per package, the number of packages per publisher, and the total number of packages in the ecosystem. If these constraints were removed, the growth would likely follow an exponential trajectory. Thirdly, the outcomes are heavily influenced by the predefined "developer_actions" parameters. In a real-world setting, developers do not check their status as consistently as the model suggests; instead, they primarily interact with the ecosystem by uploading packages for specific tasks at irregular intervals.

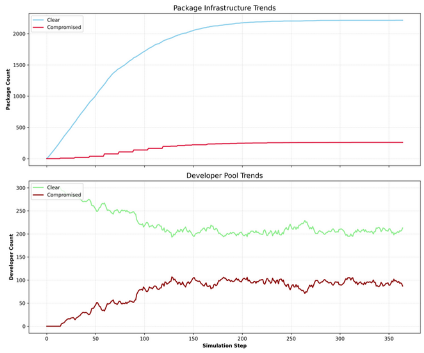

Figure 20. Trends in compromises of package and developer.

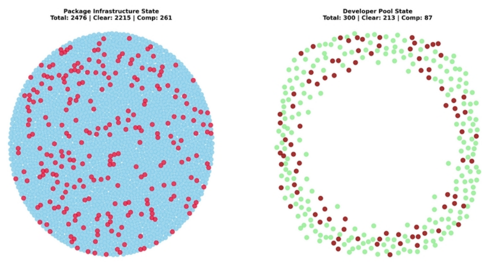

Figure 21. Integrity snapshot of the simulation.

However, this simulation is an abstract model designed to represent reality within a limited framework, consisting of an environment, defined rules, entities, specific behaviors, and a temporal structure (using either tick-based steps or event-based sequences). While simulations are essential for predictive analysis in scenarios that are too costly or dangerous to test in the real world, they are inherently prone to "drift". Without continuous recalibration, a model can rapidly diverge from reality, especially when major real-world shifts occur that the simulation cannot account for due to its predefined, static boundaries. Beyond these challenges, our simulation is subject to several other specific assumptions and limitations:

- Closed ecosystem environment - the simulation operates as a strictly closed system focused on a single repository without mirrors. Additionally, we assume the platform infrastructure itself is immutable and cannot be compromised.
- Stochasticity and randomness - event probabilities and distributions rely on pseudo-random number generation (via the Python standard library random module, which interfaces with /dev/urandom or /dev/random on Linux).
- Absence of actor confrontation - the model does not account for direct confrontations or interference between different actors, such as active conflicts between competing threat actors.
- Simplified network logistics - network-level variables, such as latency, bandwidth constraints, or connectivity issues during package uploads and downloads, are not represented.
- Scaling constraints - the total volume of packages and agents is significantly lower than the scales observed in real-world ecosystems.
- Agents have predefined roles, but in reality, they can shift between / act on multiple roles simultaneously. For example, a package publisher can do the same things as a regular user. Moreover, some developers and package publishers can be malicious like regular threat actors, but we assume that they have passively good intensions or behavior with goodwill.
- Single-project assumption - each developer or user is limited to one active project at a time. Consequently, all dependencies are grouped into a single pool rather than being isolated by specific projects or environments.
- Lack of organizational structure - the simulation does not represent organized groups, teams, or corporate entities. So, it misses the collaborative nature of departments (e.g., data science teams favoring specific libraries like numpy) and the influence of internal company policies or domain-specific dependency preferences.
- Manual vs. automated actions - the model exclusively simulates manual interactions performed by a person via a package manager toolkit. It excludes automated CI/CD pipelines, build scripts, and automated test environments that frequently pull packages from repositories.
- Direct dependency focus - the simulation only accounts for direct dependencies. Supply chain attacks via transitive dependencies are excluded, as these would require a full dependency resolution implementation. Some of these trends, specifically regarding how threat actors affect compromised package variation, are illustrated in Figure 22.

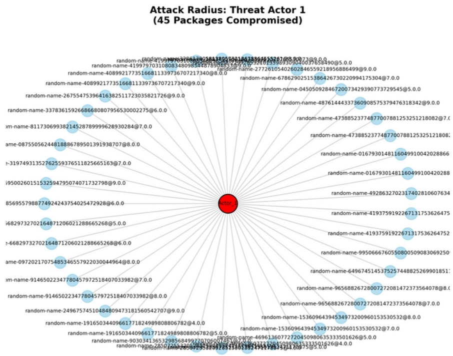

Figure 22. Example of single threat actor with a relationship to his produced compromised packages in a simulation iteration.

B. *Potential flaws, general limitations, and trade-offs of the conceptual framework*

Firstly, the comparison between the reactive and preventive models highlights distinct approaches to ecosystem security. In Model 1, a reactive environment exists without filters or entry restrictions, placing the entire burden of regulation and protection on the users. Because the platform does not intervene, users must independently manage the risks of compromise, though they can report malware after it has been discovered as a reactive measure. In contrast, Model 2 integrates both reactive and preventive strategies by introducing quality gates. These gates protect the community by temporarily blocking packages based on various factors, moving them through lifecycle states such as under consideration, public, blocked, or reported. Furthermore, every actor and package possesses both implicit and explicit trust levels. These levels represent the internal opinions of specific actors or external scores provided by the platform and community, dynamically shifting by a certain percentage based on observed compromises. By simulating a complex dynamic system across both micro and macro levels, we can analyze how these quality gates and restrictive policies effectively reduce the propagation speed of malware, vulnerabilities, and supply chain attacks. It is important to note that in a real-world environment, neither static nor dynamic preventive scanners are absolute. Because oversight is inevitable, these preventive measures must be combined with reactive approaches to create a robust, multi-layered defense.

Secondly, regarding the platform architecture, we decided to implement the analysis engine components as separate entities to facilitate concurrency across multiple instances. This design utilizes asynchronous inter-process communication to prevent performance bottlenecks. Conversely, because the implementations and contexts are often interdependent, we utilize shared libraries and common functionality to ensure cross-component compatibility.

Thirdly, sandboxed environments serve as limited models that inevitably diverge from actual production systems. Emulation and virtualization differ from physical hardware in both performance characteristics and qualitative properties, which often prevents hardware-level vulnerabilities or side-channel attacks from manifesting accurately. Consequently, sophisticated software may employ anti-VM or evasion techniques to detect isolated environments, allowing it to hide malicious behavior or cease execution entirely. While it is possible to rotate or mimic the characteristics of physical devices, real hardware continues to behave differently under specific conditions. Furthermore, certain software requires manual intervention that disrupts automation, such as patching binaries or recompiling source code. The risk of escapes from isolated environments into the broader IT infrastructure necessitates constant updates to keep the analysis environment secure. A necessity for ongoing maintenance and implementation challenges may arise, including the need to replace outdated third-party tools, integrate emerging detection techniques, and reconfigure monitoring systems to address the latest malware threats.

V. Conclusion

The management of supply chain security within critical IT infrastructure remains a vital concern, as the frequency of incidents continues to rise despite the development of new standards and tools. Given the current instability of the digital landscape, a natural shift toward preventive measures during the initial stages of software production and distribution is expected. Future work will focus on the full practical implementation of the proposed system, alongside a rigorous analysis of software components to validate the conceptual framework using real-world data and gathered statistics. We are considering a more advanced experiment utilizing an attackers vs. defenders simulation that leverages small local models. This approach aims to represent users more accurately and dynamically, providing each agent with a unique background that influences their actions and social interactions, such as discussing news or reporting issues within community groups. By hosting this within an environment of multiple virtual machines and services, we can create a much more realistic and detailed representation of reality than a simple simulation based on pseudo-random generator. However, the integration of AI agents as "Interactors" and other components currently presents reliability challenges, specifically regarding vulnerabilities like cross-prompt injection. These risks necessitate the implementation of robust guardrails to compensate for such weaknesses.

References

[1] F. Almeida, "Comparative analysis of EU-based cybersecurity skills frameworks," *Computers & Security*, vol. 151, p. 104329, Apr. 2025, doi: 10.1016/j.cose.2025.104329.

[2] M. F. Safitra, M. Lubis, and H. Fakhrurroja, "Counterattacking Cyber Threats: A Framework for the Future of Cybersecurity," *Sustainability*, vol. 15, no. 18, p. 13369, Sep. 2023, doi: 10.3390/su151813369.

[3] National Institute of Standards and Technology, "Framework for improving critical infrastructure cybersecurity, version 1.1," National Institute of Standards and Technology, Gaithersburg, MD, NIST CSWP 04162018, Apr. 2018. doi: 10.6028/NIST.CSWP.04162018.

[4] Cybersecurity and Infrastructure Security Agency (CISA), "Infrastructure resilience planning framework (IRPF)," U.S. Department of Homeland Security, Jan. 2025. Accessed: Nov. 23, 2025. [Online]. Available: https://www.cisa.gov/sites/default/files/2025-03/IRPF_3.17.2025.pdf

[5] A. M. Qatawneh and M. Al-Okaily, "The mediating role of technological vigilance between IT infrastructure and AIS efficiency," *Journal of Open Innovation: Technology, Market, and Complexity*, vol. 10, no. 1, p. 100212, Mar. 2024, doi: 10.1016/j.joitmc.2024.100212.

[6] M. Lehto, "Cyber-attacks against critical infrastructure," in *Cyber Security*, vol. 56, M. Lehto and P. Neittaanmäki, Eds., Cham: Springer International Publishing, 2022, pp. 3–42. doi: 10.1007/978-3-030-91293-2_1.

[7] G. M. Makrakis, C. Kolias, G. Kambourakis, C. Rieger, and J. Benjamin, "Industrial and critical infrastructure security: Technical analysis of real-life security incidents," *IEEE Access*, vol. 9, pp. 165295–165325, 2021, doi: 10.1109/ACCESS.2021.3133348.

[8] H. Riggs *et al.*, "Impact, vulnerabilities, and mitigation strategies for cyber-secure critical infrastructure," *Sensors*, vol. 23, no. 8, p. 4060, Apr. 2023, doi: 10.3390/s23084060.

[9] Y. Yigit *et al.*, "Critical infrastructure protection: Generative AI, challenges, and opportunities." Accessed: Nov. 05, 2025. [Online]. Available: http://arxiv.org/abs/2405.04874

[10] G. Vardakis, G. Hatzivasilis, E. Koutsaki, and N. Papadakis, "Review of smart-home security using the internet of things," *Electronics*, vol. 13, no. 16, p. 3343, Aug. 2024, doi: 10.3390/electronics13163343.

[11] J. Rauhala, "Physical weaponization of a smartphone by a third party," in *Cyber Security*, vol. 56, M. Lehto and P. Neittaanmäki, Eds., Cham: Springer International Publishing, 2022, pp. 445–460. doi: 10.1007/978-3-030-91293-2_19.

[12] D. Lindsay, S. S. Gill, D. Smirnova, and P. Garraghan, "The evolution of distributed computing systems: From fundamental to new frontiers," *Computing*, vol. 103, no. 8, pp. 1859–1878, Aug. 2021, doi: 10.1007/s00607-020-00900-y.

[13] A. Randal, "The Ideal Versus the Real: Revisiting the History of Virtual Machines and Containers," *ACM Comput. Surv.*, vol. 53, no. 1, pp. 1–31, Jan. 2021, doi: 10.1145/3365199.

[14] P. M. Mell and T. Grance, "The NIST definition of cloud computing," National Institute of Standards and Technology, Gaithersburg, MD, NIST SP 800-145, 2011. doi: 10.6028/NIST.SP.800-145.

[15] T. Arif, B. Jo, and J. H. Park, "A Comprehensive Survey of Privacy-Enhancing and Trust-Centric Cloud-Native Security Techniques Against Cyber Threats," *Sensors*, vol. 25, no. 8, p. 2350, Apr. 2025, doi: 10.3390/s25082350.

[16] Y. Alghofaili, A. Albattah, N. Alrajeh, M. A. Rassam, and B. A. S. Al-rimy, "Secure Cloud Infrastructure: A Survey on Issues, Current Solutions, and Open Challenges," *Applied Sciences*, vol. 11, no. 19, p. 9005, Sep. 2021, doi: 10.3390/app11199005.

[17] R. Ouriques, K. Wnuk, T. Gorschek, and R. B. Svensson, "The role of knowledge-based resources in Agile Software Development contexts," *Journal of Systems and Software*, vol. 197, p. 111572, Mar. 2023, doi: 10.1016/j.jss.2022.111572.

[18] P. Chuprikov, P. Eugster, and S. Mangipudi, "Security Policy as Code," *IEEE Secur. Privacy*, vol. 23, no. 2, pp. 23–31, Mar. 2025, doi: 10.1109/MSEC.2025.3535803.

[19] C. Pahl, N. Gunduz, Ö. Sezen, A. Ghamgosar, and N. El Ioini, "Infrastructure as Code: Technology Review and Research Challenges:," in *Proceedings of the 15th International Conference on Cloud Computing and Services Science*, Porto, Portugal: SCITEPRESS - Science and Technology Publications, 2025, pp. 151–158. doi: 10.5220/0013247700003950.

[20] K. Morris, *Infrastructure as code: Dynamic systems for the cloud age*, Second edition. O'Reilly, 2020.

[21] European Cyber Security Organisation (ECSO), "Technical paper: Software supply chain security," European Cyber Security Organisation, WG6, Sep. 2024. [Online]. Available: https://ecs-org.eu/ecso-uploads/2024/09/ECSO-WG6-Technical-Paper-%E2%80%94-Software-Supply-Chain-Security.pdf

[22] N. Saavedra, J. Gonçalves, M. Henriques, J. F. Ferreira, and A. Mendes, "Polyglot Code Smell Detection for Infrastructure as Code with GLITCH," in *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, Luxembourg, Luxembourg: IEEE, Sep. 2023, pp. 2042–2045. doi: 10.1109/ASE56229.2023.00162.

[23] P. R. R. Konala, V. Kumar, D. Bainbridge, and J. Haseeb, "A Framework for Measuring the Quality of Infrastructure-as-Code Scripts." Accessed: Nov. 09, 2025. [Online]. Available: http://arxiv.org/abs/2502.03127

[24] J. Sandobalin, E. Insfran, and S. Abrahao, "On the Effectiveness of Tools to Support Infrastructure as Code: Model-Driven Versus Code-Centric," *IEEE Access*, vol. 8, pp. 17734–17761, 2020, doi: 10.1109/ACCESS.2020.2966597.

[25] C. Verbruggen and M. Snoeck, "Practitioners' experiences with model-driven engineering: a meta-review," *Softw Syst Model*, vol. 22, no. 1, pp. 111–129, Feb. 2023, doi: 10.1007/s10270-022-01020-1.

[26] I. Alfonso *et al.*, "Building BESSER: An open-source low-code platform." Accessed: Nov. 09, 2025. [Online]. Available: http://arxiv.org/abs/2405.13620

[27] A. H. Chillón, J. G. Molina, J. R. Hoyos, and M. J. Ortín, "Propagating Schema Changes to Code: An Approach Based on a Unified Data Model," in *Proceedings of the workshops of the EDBT/ICDT 2023 joint conference*, CEUR-WS, Mar. 2023. [Online]. Available: https://ceur-ws.org/Vol-3379/CoMoNoS_2023_id251_Alberto_Hernandez_Chillon.pdf

[28] J. Di Rocco, D. Di Ruscio, C. Di Sipio, P. T. Nguyen, and R. Rubei, "On the use of large language models in model-driven engineering," *Softw Syst Model*, vol. 24, no. 3, pp. 923–948, Jun. 2025, doi: 10.1007/s10270-025-01263-8.

[29] I. Stoica *et al.*, "Specifications: The missing link to making the development of LLM systems an engineering discipline." Accessed: Nov. 09, 2025. [Online]. Available: http://arxiv.org/abs/2412.05299

[30] S. Shankar, J. D. Zamfirescu-Pereira, B. Hartmann, A. Parameswaran, and I. Arawjo, "Who Validates the Validators? Aligning LLM-Assisted Evaluation of LLM Outputs with Human Preferences," in *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology*, Pittsburgh PA USA: ACM, Oct. 2024, pp. 1–14. doi: 10.1145/3654777.3676450.

[31] *OWASP/www-project-top-10-infrastructure-security-risks*. (Oct. 22, 2025). OWASP. Accessed: Nov. 05, 2025. [Online]. Available: https://github.com/OWASP/www-project-top-10-infrastructure-security-risks

[32] A. Sabetta *et al.*, "Known Vulnerabilities of Open Source Projects: Where Are the Fixes?," *IEEE Secur. Privacy*, vol. 22, no. 2, pp. 49–59, Mar. 2024, doi: 10.1109/MSEC.2023.3343836.

[33] Cybersecurity and Infrastructure Security Agency (CISA), "A Shared Vision of Software Bill of Materials (SBOM) for Cybersecurity," U.S. Department of Homeland Security, Sep. 2025. Accessed: Nov. 23, 2025. [Online]. Available: https://www.cisa.gov/sites/default/files/2025-09/joint-guidance-a-shared-vision-of-software-bill-of-materials-for-cybersecurity_508c.pdf

[34] L. A. Jaatun, S. Marie Sørlien, R. Borgaonkar, S. Taylor, and M. G. Jaatun, "Software Bill of Materials in Critical Infrastructure," in *2023 IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*, Naples, Italy: IEEE, Dec. 2023, pp. 319–324. doi: 10.1109/CloudCom59040.2023.00059.

[35] J. Klein and D. Reynolds, "Infrastructure as Code: Final Report," Carnegie Mellon University, Dec. 2018. [Online]. Available: https://www.sei.cmu.edu/documents/576/2019_019_001_539335.pdf

[36] S. S. Ragavan, M. Codoban, D. Piorkowski, D. Dig, and M. Burnett, "Version Control Systems: An Information Foraging Perspective," *IIEEE Trans. Software Eng.*, vol. 47, no. 8, pp. 1644–1655, Aug. 2021, doi: 10.1109/TSE.2019.2931296.

[37] The MITRE Corporation, "Supply Chain Compromise, Technique T1195 - Enterprise | MITRE ATT&CK®." Accessed: Nov. 26, 2025. [Online]. Available: https://attack.mitre.org/versions/v18/techniques/T1195/

[38] C. Okafor, T. R. Schorlemmer, S. Torres-Arias, and J. C. Davis, "SoK: Analysis of Software Supply Chain Security by Establishing Secure Design Properties," in *Proceedings of the 2022 ACM Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses*, Los Angeles CA USA: ACM, Nov. 2022, pp. 15–24. doi: 10.1145/3560835.3564556.

[39] S. Neupane, G. Holmes, E. Wyss, D. Davidson, and L. D. Carli, "Beyond typosquatting: An in-depth look at package confusion," in *32nd USENIX security symposium (USENIX security 23)*, Anaheim, CA: USENIX Association, Aug. 2023, pp. 3439–3456. [Online]. Available: https://www.usenix.org/conference/usenixsecurity23/presentation/neupane

[40] M. A. Sasse, M. Smith, C. Herley, H. Lipford, and K. Vaniea, "Debunking Security-Usability Tradeoff Myths," *IEEE Secur. Privacy*, vol. 14, no. 5, pp. 33–39, Sep. 2016, doi: 10.1109/MSP.2016.110.

[41] R. Sen, J. Choobineh, and S. Kumar, "Determinants of Software Vulnerability Disclosure Timing," *Production and Operations Management*, vol. 29, no. 11, pp. 2532–2552, Nov. 2020, doi: 10.1111/poms.13120.

[42] A. J. Jafari, D. E. Costa, A. Abdellatif, and E. Shihab, "Dependency Practices for Vulnerability Mitigation." Accessed: Nov. 05, 2025. [Online]. Available: http://arxiv.org/abs/2310.07847

[43] M. Alfadel, D. E. Costa, and E. Shihab, "Empirical Analysis of Security Vulnerabilities in Python Packages," in *2021 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, Honolulu, HI, USA: IEEE, Mar. 2021, pp. 446–457. doi: 10.1109/SANER50967.2021.00048.

[44] J. Dietrich, S. Rasheed, A. Jordan, and T. White, "On the Security Blind Spots of Software Composition Analysis." Accessed: Nov 05, 2025. [Online]. Available: http://arxiv.org/abs/2306.05534

[45] W. Enck and L. Williams, "Top Five Challenges in Software Supply Chain Security: Observations From 30 Industry and Government Organizations," *IEEE Secur. Privacy*, vol. 20, no. 2, pp. 96–100, Mar. 2022, doi: 10.1109/MSEC.2022.3142338.

[46] M. Böhme, E. Bodden, T. Bultan, C. Cadar, Y. Liu, and G. Scanniello, "Software Security Analysis in 2030 and Beyond: A Research Roadmap," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 5, pp. 1–26, Jun. 2025, doi: 10.1145/3708533.

[47] N. Zaidenberg, M. Kiperberg, and A. Resh, "TrulyProtect—Virtualization-Based Protection Against Reverse Engineering," in *Cyber Security*, vol. 56, M. Lehto and P. Neittaanmäki, Eds., Cham: Springer International Publishing, 2022, pp. 353–366. doi: 10.1007/978-3-030-91293-2_15.

[48] A. Iklody, G. Wagener, A. Dulaunoy, S. Mokaddem, and C. Wagner, "Decaying Indicators of Compromise." arXiv, Mar. 2018. doi: 10.48550/arXiv.1803.11052.

[49] O. Or-Meir, N. Nissim, Y. Elovici, and L. Rokach, “Dynamic Malware Analysis in the Modern Era—A State of the Art Survey,” *ACM Comput. Surv.*, vol. 52, no. 5, pp. 1–48, Sep. 2020, doi: 10.1145/3329786.
[50] Williams A. D., “Enabling Global Collaboration,” The Linux Foundation, Jan. 2025. [Online]. Available: https://project.linuxfoundation.org/hubfs/LF%20Research/Overcoming%20OS%20Fragmentation%20-%20Report.pdf
[51] M. Mirakhorli *et al.*, “A Landscape Study of Open Source and Proprietary Tools for Software Bill of Materials (SBOM).” Accessed: Nov. 23, 2025. [Online]. Available: https://arxiv.org/abs/2402.11151
[52] N. Wintersgill, T. Stalnaker, L. A. Heymann, O. Chaparro, and D. Poshyvanyk, “‘The Law Doesn’t Work Like a Computer’: Exploring Software Licensing Issues Faced by Legal Practitioners,” *Proc. ACM Softw. Eng.*, vol. 1, no. FSE, pp. 882–905, Jul. 2024, doi: 10.1145/3643766.
[53] D. Bozzini, “The regimes of ethical hacking: Moral projects and the emergence of a market for vulnerability,” *Information, Communication & Society*, pp. 1–18, May 2025, doi: 10.1080/1369118X.2025.2498683.
[54] J. Arshad, M. Talha, B. Saleem, Z. Shah, H. Zaman, and Z. Muhammad, “A Survey of Bug Bounty Programs in Strengthening Cybersecurity and Privacy in the Blockchain Industry,” *Blockchains*, vol. 2, no. 3, pp. 195–216, Jul. 2024, doi: 10.3390/blockchains2030010.
[55] A. S. Abdiraman, L. S. Aldasheva, B. Darmenov, T. I. Omurzakov, and A. B. Zakirova, “Comparative analysis of application platform for learning cybersecurity through the Capturing the Flag Competitions,” *bultechenukz*, vol. 145, no. 4, pp. 49–57, Dec. 2023, doi: 10.32523/2616-7263-2023-145-4-49-57.
[56] W. K. G. Assunção, L. Marchezan, L. Arkoh, A. Egyed, and R. Ramler, “Contemporary Software Modernization: Strategies, Driving Forces, and Research Opportunities,” *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 5, pp. 1–35, Jun. 2025, doi: 10.1145/3708527.
[57] C. Pohl and H.-J. Hof, “Secure Scrum: Development of Secure Software with Scrum,” Jul. 10, 2015, *arXiv*: arXiv:1507.02992. doi: 10.48550/arXiv.1507.02992.
[58] P. Mell, J. Shook, and R. Harang, “Measuring and Improving the Effectiveness of Defense-in-Depth Postures,” in *Proceedings of the 2nd Annual Industrial Control System Security Workshop*, Los Angeles CA USA: ACM, Dec. 2016, pp. 15–22. doi: 10.1145/3018981.3018986.
[59] S. Berhe, M. Maynard, and F. Khomh, “Maintenance Cost of Software Ecosystem Updates,” *Procedia Computer Science*, vol. 220, pp. 608–615, 2023, doi: 10.1016/j.procs.2023.03.077.
[60] P. Dehraj and A. Sharma, “A review on architecture and models for autonomic software systems,” *J Supercomput*, vol. 77, no. 1, pp. 388–417, Jan. 2021, doi: 10.1007/s11227-020-03268-0.
[61] R. Sammak, A. L. Rotthaler, H. S. Ramulu, D. Wermke, and Y. Acar, “Developers’ Approaches to Software Supply Chain Security: An Interview Study,” in *Proceedings of the 2024 Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses*, Salt Lake City UT USA: ACM, Nov. 2023, pp. 56–66. doi: 10.1145/3689944.3696160.
[62] R. M. Tsoupidi, E. Troubitsyna, and P. Papadimitratos, “Protecting Cryptographic Libraries Against Side-Channel and Code-Reuse Attacks,” *IEEE Secur. Privacy*, vol. 23, no. 5, pp. 46–55, Sep. 2025, doi: 10.1109/MSEC.2024.3498736.
[63] P. Ladisa, H. Plate, M. Martinez, and O. Barais, “Taxonomy of Attacks on Open-Source Software Supply Chains.” Accessed: Dec. 01, 2025. [Online]. Available: http://arxiv.org/abs/2204.04008
[64] A. Singer and M. Bishop, “Trust-Based Security; Or, Trust Considered Harmful,” in *New Security Paradigms Workshop 2020*, Online USA: ACM, Oct. 2020, pp. 76–89. doi: 10.1145/3442167.3442179.
[65] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, “Zero Trust Architecture,” National Institute of Standards and Technology, Aug. 2020. doi: 10.6028/NIST.SP.800-207.
[66] T. Babenko, S. Amanzholova, R. Lisnevskyi, and A. Abylgazy, “Cybersecurity-level assessment models,” in *DTESI 2024: 9th international conference on digital technologies in education, science and industry*, CEUR-WS, Oct. 2024. [Online]. Available: https://ceur-ws.org/Vol-3966/W4Paper2.pdf
[67] Cybersecurity and Infrastructure Security Agency (CISA), “The Journey to Zero Trust: Microsegmentation in Zero Trust Part One: Introduction and Planning,” U.S. Department of Homeland Security, Jul. 2025. [Online]. Available: https://www.cisa.gov/sites/default/files/2025-07/ZT-Microsegmentation-Guidance-Part-One_508c.pdf
[68] Z. A. Collier and J. Sarkis, “The zero trust supply chain: Managing supply chain risk in the absence of trust,” *International Journal of Production Research*, vol. 59, no. 11, pp. 3430–3445, Jun. 2021, doi: 10.1080/00207543.2021.1884311.
[69] U.S. Department of Defense, Chief Information Officer (DoD CIO), “DoD enterprise DevSecOps fundamentals, v2.5,” U.S. Department of Defense, v2.5, Oct. 2024. [Online]. Available: https://dodcio.defense.gov/Portals/0/Documents/Library/DoD%20Enterprise%20DevSecOps%20Fundamentals%20v2.5.pdf
[70] S. Mushtaq, M. Mohsin, and M. M. Mushtaq, “A Systematic Literature Review on the Implementation and Challenges of Zero Trust Architecture Across Domains,” *Sensors*, vol. 25, no. 19, p. 6118, Oct. 2025, doi: 10.3390/s25196118.
[71] H. Bangui, M. Ge, and B. Buhnova, “When Trustless meets Trust: Blockchain Consensus Review and Reconsideration,” *Procedia Computer Science*, vol. 246, pp. 3351–3360, 2024, doi: 10.1016/j.procs.2024.09.222.
[72] L. Grilli and P. Speziali, “Combining Git and Blockchain for Trusted Information Sharing,” *IEEE Access*, vol. 12, pp. 88383–88409, 2024, doi: 10.1109/ACCESS.2024.3418749.
[73] Anthropic, “Disrupting the first reported AI-orchestrated cyber espionage campaign,” Anthropic, Nov. 2025. [Online]. Available: https://assets.anthropic.com/m/ec212e6566a0d47/original/Disrupting-the-first-reported-AI-orchestrated-cyber-espionage-campaign.pdf
[74] N. Tendikov, "aitu-sist-2026_research-paper," GitHub repository, Mar. 22, 2026. [Online]. Available: https://github.com/NoyanTM/aitu-sist-2026_research-paper. [Accessed: Apr. 3, 2026].